

Rechnen mit ganzen Zahlen

Alisa Mirena
alisa.mirena@uni-bielefeld.de

28.04.2026
Proseminar: Kombinatorik
Universität Bielefeld

Contents

1	Addition und Subtraktion	3
2	Multiplikation	3
3	Euklidischer Algorithmus	6

1 Addition und Subtraktion

Der Algorithmus hat Laufzeit $O(l)$, wobei l die maximale Anzahl an Stellen der beteiligten Zahlen ist. $O(l)$ wächst proportional, d.h. wenn die Zahlen mit denen man rechnet doppelt so lang werden, dann wird auch die Rechnung doppelt so lang.

Proposition 1.1

Für zwei ganze Zahlen x und y können die Summe $x+y$ und die Differenz $x-y$ in Laufzeit $O(l)$ berechnet werden, wobei

$$l = 1 + \lfloor \log_2(\max\{|x|, |y|, 1\}) \rfloor$$

.

2 Multiplikation

Für die Multiplikation und Division gilt die Laufzeit $O(l^2)$, auch quadratische Laufzeit genannt. Der russische Mathematiker Karatsuba fand im Jahr 1960 als Erster eine Möglichkeit, um Multiplikation noch schneller durchzuführen. Seine Idee war es, die l -stelligen zu multiplizierenden Zahlen x und y in zwei $l/2$ -stellige Zahlen zu zerlegen. Damit werden x und y wie folgt dargestellt: $x = x'B + x''$ und $y = y'B + y''$. B ist hierbei die Potenz der Basis. Um das gesamte Produkt xy zu erhalten, reicht es die einzelnen Produkte $[p := x'y']$, $[q := x''y'']$ und $[r := (x' + x'')(y' + y'')]$ zu berechnen.

$$x \times y = (x'y')B^2 + (x'y'' + x''y')B + x''y'' = pB^2 + (r - p - q)B + q. \quad (2.1)$$

Beispiel

Sei $x = 1234$ und $y = 5678$.

$$x' = 12 \text{ und } x'' = 34; \quad y' = 56 \text{ und } y'' = 78$$

$$p := 12 \times 56$$

$$q := 34 \times 78$$

$$r := (12 + 34) \times (56 + 78).$$

$$x \times y = (12 \times 56) \times 100^2 + (12 \times 78 + 34 \times 56) \times 100 + 34 \times 78 =$$

$$(12 \times 56) \times 100^2 + ((12 + 34) \times (56 + 78) - 12 \times 56 - 34 \times 78) \times 100 + 34 \times 78 =$$

$$7006654 = 1234 \times 5678$$

Das zeigt, dass wir nur 3 einzelne Multiplikationen(r,p,q) durchführen müssen, um das Ergebnis von $x \times y$ berechnen zu können.

Beweis von 2.1

$$\textit{Proof. } x \times y = (x'B + x'') \times (y'B + y'') = x'y' + x'y''B + x''y'B + x''y'' =$$

$$(x'y')B^2 + (x'y'' + x''y')B + x''y'' = pB^2 + (r - p - q)B + q.$$

□

Karatsuba Algorithmus 2.2

Karatsuba-Algorithmus

$x, y \in N$ in Binärdarstellung

$x \cdot y$ in Binärdarstellung

$x < 8$ und $y < 8$ $x \cdot y$ $l \leftarrow 1 + \lfloor \log_2(\max\{x, y\}) \rfloor$

$k \leftarrow \lfloor l/2 \rfloor$

$B \leftarrow 2^k$

$x' \leftarrow \lfloor x/B \rfloor, x'' \leftarrow x \bmod B$

$y' \leftarrow \lfloor y/B \rfloor, y'' \leftarrow y \bmod B$

$p \leftarrow x' \cdot y'$ (rekursiv)

$q \leftarrow x'' \cdot y''$ (rekursiv)

$r \leftarrow (x' + x'')(y' + y'')$ (rekursiv)

$pB^2 + (r - p - q)B + q$

- a) Falls $x, y < 8$, dann wird die einfache Multiplikation genutzt.
- b) Falls $x, y > 8$, dann wird Karatsubas Algorithmus benutzt.

(Falls x und y nach der Teilung immernoch groß sind, wird der Algorithmus wieder verwendet, bis die Zahlen klein genug sind)

Satz 2.3

Die Multiplikation zweier in Binärdarstellung gegebener natürlicher Zahlen x und y ist nach der Schulmathematik in Laufzeit $O(l^2)$ möglich. Nach dem Karatsuba-Algorithmus geht dies deutlich schneller. Demnach gilt Laufzeit $O(l^{\log_2 3})$. Wobei weiterhin gilt: $l = 1 + \lfloor \log_2(\max\{|x|, |y|, 1\}) \rfloor$.

Beispiel

$$\begin{aligned} l &= 1 + \lfloor \log_2(\max\{|1234|, |5678|, 1\}) \rfloor = 14 \\ O(l^2) &= 14^2 = 196 \text{ (Rechenschritte)} \\ O(l^{\log_2 3}) &= 14^1,585 = 65,65 \end{aligned}$$

Beweis von 2.3

Proof. z.z.: $T(l) = O(l^{\log_2 3})$

$T(l)$:= Laufzeit in Abhängigkeit von Bitlänge l .

Fall 1.: $l \leq 3, : T(l) \leq c$

Fall 2.: $l \geq 4 : T(l) \leq cl + 3T(\lceil \frac{l}{2} \rceil + 1)$

Beweis per Induktion:

Setze $l := 2^m + 2$

I.B.: $T(2^m + 2) \leq c(4 \times 3^m - 2 \times 2^m - 1)$ für alle $m \in \mathbf{N}_0$.

I.A.: $m = 0$.

$l = 2^0 + 2 = 3$. Nach Fall 1 gilt: $T(3) \leq c$

I.V.: Ang. $T(2^{m-1} + 2) \leq c(4 \times 3^{m-1} - 2 \times 2^{m-1} - 1)$.

I.S.: von $(m-1)$ nach m .

$$T(2^m + 2) \leq c(2^m + 2) + 3T(\lceil \frac{2^m + 2}{2} \rceil + 1)$$

Da $\frac{2^m + 2}{2} + 1 = 2^{m-1} + 1$, können wir vereinfachen zu:

$$\begin{aligned}
T(2^m + 2) &\leq c(2^m + 2) + 3T(2^{m-1} + 2) \\
(\text{nach I.V.}) &\Leftrightarrow T(2^m + 2) \leq +3(c(4 \times 3^{m-1} - 2 \times 2^{m-1} - 1)) \\
&= T(2^m + 2) \leq c(2^m + 2) + c \times 12 \times 3^{m-1} - c \times 6 \times 2^{m-1} - c \times 3 \\
&= T(2^m + 2) \leq (2^m + 2 + 12 \times 3^{m-1} - 6 \times 2^{m-1} - 3) \\
&= T(2^m + 2) \leq c(2^m + 4 \times 3^m - 3 \times 2^m - 1) \\
&= T(2^m + 2) \leq c(4 \times 3^m - 2 \times 2^m - 1)
\end{aligned}$$

Schlussfolgerung: Setze $m := \log_2 l$.

Dann erhalten wir:

$$T(l) \leq c \times 3^m = c \times 3^{\log_2 l} = c \times l^{\log_2 3}$$

.

□

3 Euklidischer Algorithmus

Der Euklidische Algorithmus, beschrieben von Euklid um 300 v.Chr. in Buch VII seiner Elemente, erlaubt das effiziente Berechnen des größten gemeinsamen Teilers zweier Zahlen, und damit das Kürzen von Brüchen.

Definition 3.1

Seien $a, b \in \mathbb{N}$.

$\text{ggT}(a, b)$ ist der größte gemeinsame Teiler von a und b ; d.h. die größte natürliche Zahl, die sowohl a als auch b ganzzahlig teilt.

Es gelten folgende Eigenschaften:

- 1) Gilt $\text{ggT}(a, b) = 1$, so heißen a und b teilerfremd.
- 2) $\text{ggT}(a, 0) := \text{ggT}(0, a) := a \quad \forall a \in \mathbb{N}$.

Definition 3.2

Seien $a, b \in \mathbb{N}$.

$\text{kgV}(a, b)$ ist das kleinste gemeinsame Vielfache von a und b ; d.h. die kleinste natürliche Zahl, von der sowohl a als auch b Teiler sind.

Lemma 3.3

- a) $a \times b = \text{ggT}(a, b) \times \text{kgV}(a, b)$
- b) $\text{ggT}(a, b) = \text{ggT}(a \bmod b, b)$

Beispiel

Sei $a = 10$ und $b = 8$

$$\text{ggT}(a, b) = 2$$

$$\text{kgV}(a, b) = 40$$

$$\text{a) } a \times b = 10 \times 8 = 80 = 2 \times 40 = \text{ggT}(a, b) \times \text{kgV}(a, b)$$

$$\text{b) } 2 = \text{ggT}(10, 8) = \text{ggT}(10 \bmod 8, 8) = \text{ggT}(2, 8)$$

Beweis von Lemma 3.3

Proof. a) 1. z.z. $\frac{ab}{\text{ggT}(a, b)} \geq \text{kgV}(a, b)$

Sei $\text{ggT}(a, b) := x$

$$\frac{ab}{x} = a \left(\frac{b}{x} \right) = b \left(\frac{a}{x} \right)$$

Da $x = \text{ggT}(a, b)$ ist, muss x ein ganzzahliger Teil von b (bzw. a) sein, weshalb auch $\frac{b}{x}$ ganzzahlig ist. Damit ist $\frac{ab}{x}$ ein Vielfaches von a und b . Da $\frac{ab}{x}$ ein gemeinsames Vielfaches von a und b ist, muss es größer oder gleich dem kleinsten gemeinsamen Vielfachen sein.

Damit folgt: $\frac{ab}{\text{ggT}(a, b)} \geq \text{kgV}(a, b)$

(Erinnerung: M ist ein Vielfaches von a , falls eine natürliche Zahl x existiert, s.d. $a \times x = M$)

$$\text{2. z.z. } \frac{ab}{\text{kgV}(a, b)} \leq \text{ggT}(a, b)$$

Da $\text{kgV}(a,b)$ ein Vielfaches von a ist, so exist. ein $m \in \mathbf{N}$, s.d. gilt:
 $\text{kgV}(a,b) = a \times m$.

$$\frac{ab}{\text{kgV}(a,b)} = \frac{ab}{am} = \frac{b}{m}$$

Damit ist $\frac{ab}{\text{kgV}(a,b)}$ ein Teiler von b . (a ist analog)

Da $\frac{ab}{\text{kgV}(a,b)}$ ein gemeinsamer Teiler von a und b ist, muss $\frac{ab}{\text{kgV}(a,b)}$ kleiner oder gleich dem $\text{ggT}(a,b)$ sein.

b) z.z. $\text{ggT}(a,b) = \text{ggT}(a \bmod b, b)$

$$a = b \times q + r \iff r = a - b \times q$$

Sei d ein Teiler von a und b . Dann gilt: $d|a \iff d|bq + r \iff d|a - bq \iff d|r$

Damit folgt: $\text{ggT}(a,b) = \text{ggT}(r,b) = \text{ggT}(a \bmod b, b)$

□

Algorithmus 3.4

Algorithmus (Euklidischer Algorithmus)

Eingabe: $a, b \in \mathbf{N}$

Ausgabe: $\text{ggT}(a,b)$

while $a > 0$ und $b > 0$ **do**

if $a < b$ **then** $b \leftarrow b \bmod a$

else $a \leftarrow a \bmod b$

output $\max\{a,b\}$

Beispiel

$a = 48, b = 18$

$$\begin{aligned} \text{ggT}(48, 18) &= \text{ggT}(48 \bmod 18, 18) = \text{ggT}(12, 18) = \text{ggT}(12, 6) = \\ &\text{ggT}(0, 6) = 6 \end{aligned}$$

Programm 3.8 (Euklidischer Algorithmus)

```
1 // euclid.cpp (Euclidean Algorithm)
2
3 #include <iostream>
4
5 using myint = long long;
6
7 myint gcd(myint a, myint b)           // compute greatest common divisor
8 {                                     // using Euclidean algorithm
9     if (b == 0) {
10         return a;
11     }
12     else {
13         return gcd(b, a % b);
14     }
15 }
16
17
18 int main()
19 {
20     myint a, b;
21     std::cout << "This program computes the greatest common divisor.\n"
22               << "Enter two natural numbers, separated by blank: ";
23     std::cin >> a >> b;
24     std::cout << "gcd(" << a << ", " << b << ") = " << gcd(a,b) << "\n";
25 }
```

Formel für Laufzeit:

$$2 + \lfloor \log_2(\max\{a, 1\}) \rfloor + \lfloor \log_2(\max\{b, 1\}) \rfloor$$

Definition 3.5: Fibonacci-Zahlen

Für eine genauere Analyse der Laufzeit des Euklidischen Algorithmus benötigen wir die Fibonacci Zahlen. Sie sind benannt nach Leonardo Fibonacci, welcher damit das Wachstum einer Kaninchenpopulation beschreibt. Doch schon vor mehr als 400 Jahren vor Fibonacci waren diese Zahlen schon in Indien bekannt (ca. 1170-1240).

F_i ist für $i = 0, 1, 2, 3, 4, \dots$ gegeben durch:

$$F_0 := 0$$

$$F_1 := 1$$

$$F_{n+1} := F_n + F_{n-1} \text{ für } n \in \mathbb{N}$$

Die ersten Fibonacci Zahlen sehen wie folgt aus: $\{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55\}$
D.h. jede Zahl ist die Summe der beiden vorherigen Zahlen.

Lemma 3.6: Movire-Binet-Formel

$\forall n \in (N_0)$ ist:

$$F_n = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right)$$

Eine explizite Formel zur Berechnung der n-ten Fibonacci Zahl F_n , ohne die vorherigen Zahlen berechnen zu müssen.

Beweis a

Proof. Eine Fibonacci Zahl ist die Summe ihrer beiden Vorgänger. Es gilt:

$$f(n+1) = f(n) + f(n-1).$$

Ansatz: $f(n) = x^n$

$$x^{n+1} = x^n + x^{n-1} \mid x^{n-1}$$

$$\Leftrightarrow x^2 = x + 1$$

$$\Leftrightarrow x^2 - x - 1 = 0$$

$$\Rightarrow x_1 = \frac{1+\sqrt{5}}{2}, x_2 = \frac{1-\sqrt{5}}{2}$$

OBdA: $f(n) = Ax_1^n + Bx_2^n$, wobei A und B zwei beliebige unbekannte Konstanten sind mit $A, B \neq 0$.

Um herauszufinden, was A und B sind, setzen wir $n=0$ und $n=1$ ein.

$$\text{Da } f(0) = 0 : f(0) = A + B = 0 \Rightarrow B = -A$$

$$\text{Da } f(1) = 1 : f(1) = Ax_1 + Bx_2 = 1 \Rightarrow Ax_1 - Ax_2 = 1$$

$$\Leftrightarrow A \frac{1+\sqrt{5}}{2} - A \frac{1-\sqrt{5}}{2} = 1$$

$$\Rightarrow A = \frac{1}{\sqrt{5}} \text{ und } B = -\frac{1}{\sqrt{5}}$$

Damit erhalten wir die gesuchte Formel:

$$f(n) = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right)$$

.

Beweis b: Per Induktion

$$\text{I.A. } F_0 = f(0) = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^0 - \left(\frac{1-\sqrt{5}}{2} \right)^0 \right) = \frac{1}{\sqrt{5}} (1 - 1) = 0$$

$$F_1 = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^1 - \left(\frac{1-\sqrt{5}}{2} \right)^1 \right) = \frac{1}{\sqrt{5}} \left(\frac{2\sqrt{5}}{2} \right) = 1$$

I.V. von $n-1$ nach $n-2$

$$\begin{aligned} F_n &= F_{n-1} + F_{n-2} \\ &= \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^{n-1} + \left(\frac{1+\sqrt{5}}{2} \right)^{n-2} - \left(\frac{1-\sqrt{5}}{2} \right)^{n-1} - \left(\frac{1-\sqrt{5}}{2} \right)^{n-2} \right) \\ &= \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^{n-2} \left(\frac{1+\sqrt{5}}{2} + 1 \right) - \left(\frac{1-\sqrt{5}}{2} \right)^{n-2} \left(\frac{1-\sqrt{5}}{2} + 1 \right) \right) \\ &= \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right) \end{aligned}$$

□

Lemma 3.7

Falls $a > b > 0$ und der Algorithmus 3.4 $k \geq 1$ Iterationen der while-Schleife durchläuft, so gilt:

$$a \geq F_{k+2} \text{ und } b \geq F_{k+1}$$

Besipiel

Ang. wir benötigen 3 Schritte, dann muss $b \geq 3 = F_{3+1}$ sein und $a \geq 5 = F_{3+2}$

$$ggT(a, b) = ggT(5, 3) = ggT(3, 2) = ggT(2, 0) = 2$$

Beweis von 3.7

Proof. Per Induktion:

Ang. die Aussage gilt für $(k-1)$ Iterationen

I.V.: $a \bmod b \geq F_{k+1}$, $b \geq F_k$

I.A.: $k=1$: $b \geq F_{1+1} = 1$

$$a \geq F_{1+2} = 2$$

Da $a > b > 0$, und $b \in \mathbf{N}$, muss $b \geq 1$ sein und damit $a \geq 2$

I.S.: von $(k-1)$ nach k .

Nach der ersten Iteration arbeitet der Algorithmus mit den Zahlen $(b, a \bmod b)$. Da insgesamt k Iterationen stattfinden, durchläuft der Algorithmus für dieses neue Zahlenpaar noch $k-1$ Iterationen. Daher können wir die Induktionsvoraussetzung anwenden und erhalten:

$$a \bmod b \geq F_k, \quad b \geq F_{k+1}$$

.

$$\text{Es gilt: } a = \lfloor \frac{a}{b} \rfloor \times b + (a \bmod b)$$

$$\text{Da } a > b : \lfloor \frac{a}{b} \rfloor \geq 1$$

Damit folgt:

$$a = \lfloor \frac{a}{b} \rfloor \times b + (a \bmod b) \geq b + (a \bmod b) \geq F_{k+1} + F_k = F_{k+2}$$

.

□

Satz von Lamé 3.8

Falls $a \geq b$ und $b < F_{k+1}$ für ein $k \in \mathbf{N}$, so durchläuft der Algorithmus 3.4 weniger als k Iterationen der while-Schleife.

Beweis von 3.8

Fall 1: Falls $b = 0$: Algorithmus durchläuft gar keine Iterationen, d.h. $k = 0$. Denn $ggT(a, b) = ggT(a, 0) = a$.

Fall 2: Falls $a = b$: Algorithmus durchläuft nur eine Iteration. Denn: $ggT(a, b) = ggT(a, a) = ggT(a, a \bmod a) = ggT(a, 0) = a$.

Korollar 3.9

Falls $a \geq b > 1$, so durchläuft der Algorithmus 3.4 höchstens $\lceil \log_\varphi b \rceil$ Iterationen, wobei $\varphi = \frac{1+\sqrt{5}}{2}$ (Goldener Schnitt). Die Gesamtzeit des Algorithmus zur Berechnung von $ggT(a, b)$ ist $= O((\log a)^3)$.

Beweis von 3.9

Proof. Teil 1: Beweis per Induktion über n.

$$\text{I.B.: } F_n > \varphi^{n-2}$$

I.A.: $n = 3$:

$$F_3 = 2 > \varphi^{3-2} = \varphi^1$$

$$\text{I.S.: } F_n > \varphi^{n-2}, F_{n-1} > \varphi^{n-3}$$

Dann ist: $F_{n+1} = F_n + F_{n-1}$

$$\Rightarrow F_{n+1} > \varphi^{n-2} + \varphi^{n-3} = \varphi^{n-3}(\varphi + 1) = \varphi^{n-3} \times \varphi^2$$

$$\text{Trick : } 1 + \varphi = \varphi^2$$

$$\text{Einsetzten: } F_{n+1} > \varphi^{n-3} \times \varphi^2$$

Teil 2:

$$\text{Setze } k = 1 + \lceil \log_\varphi b \rceil \geq 2$$

$$\text{z.z. } b \leq \varphi^{k-1} < F_{k+1}$$

Gilt nach Satz von Lame.

Teil 3:

Computer machte etwa $\log b$ Iterationen. Da $a \geq b : O(\log a)$:

Division mit Schulmethode dauert bei n-stelligen Zahlen etwa $O(n^2)$.

Also haben wir demnach: $O((\log a)^2)$

$$\text{Anzahl Iterationen} \times \text{Anzahl Rechenschritte pro Iteration} = \\ O(\log a) \times O((\log a)^2) = O((\log a)^3)$$

□

Literaturverzeichnis

Hougardy, S. und Vygen, J.: Algorithmische Mathematik 2., korrigierte und erweiterte Auflage. Berlin: Springer Spektrum, 2018.